

Ordered Multi-Signatures from the DL Assumption

Keisuke Hara^{1,2}, Keisuke Tanaka³, and Masayuki Tezuka^{4,3}

¹ The University of Osaka, Osaka, Japan

² National Institute of Advanced Industrial Science and Technology, Tokyo, Japan

³ Institute of Science Tokyo, Tokyo, Japan

⁴ Nagoya City University, Nagoya, Japan

`hara-keisuke@ist.osaka-u.ac.jp`, `keisuke@is.titech.ac.jp`,
`m.tezuka@nsc.nagoya-cu.ac.jp`

Abstract. Ordered multi-signatures allow multiple signers to sign a common message sequentially, and anyone to verify the signing order of signers with a public-key list. Recently, Baum et al. (PKC 2025) proposed an ordered multi-signature scheme over a pairing-free group by modifying the multi-signature scheme MuSig2 by Nick et al. (CRYPTO 2021). The security of their ordered multi-signature scheme was proven under the algebraic one-more discrete logarithm (AOMDL) assumption in the random oracle model (ROM). The AOMDL assumption is stronger than the discrete logarithm (DL) assumption. To strengthen the assurance of security, it is desirable either to prove their scheme under the DL assumption or give a construction whose security is proven under the DL assumption.

In this paper, we give an ordered multi-signature scheme from linear hash function families. Our scheme is obtained by generalizing Baum et al.'s scheme via linear hash function families. The security of our scheme is proven under the algebraic one-more preimage resistance (AOMPR) property of a linear hash function in the ROM. There is a linear hash function whose AOMPR property can be proven under the DL assumption. Thus, by using this linear hash function, we obtain the first DL-based ordered multi-signature scheme.

1 Introduction

1.1 Background

Multi-Signatures. Multi-signatures by Itakura and Nakamura [19] allow a set of signers to sign a common message. The size of a resulting multi-signature should be independent of the number of signers. Multi-signatures have applications in blockchain to reduce the transaction size and the verification cost by aggregating multiple signatures into a compact signature. Multi-signatures are classified into *non-interactive* and *interactive* multi-signatures.

Non-interactive multi-signatures require only a single round of communication among a set of signers to produce a compact signature σ . That is, each

signer i generates a signature σ_i on a message m without interacting with other signers and sends it to a combiner. The combiner compresses the signatures $\sigma_1, \dots, \sigma_n$ for the common message m into a short signature σ . To date, while we have a lot of non-interactive multi-signature schemes over pairing groups (e.g., [11,2,37,13]), giving pairing-free non-interactive multi-signature schemes is still a difficult task.

Interactive multi-signatures require an interaction during the run of the signing protocol. So far, we have some three-round interactive multi-signature schemes over pairing-free groups [9,26,18]. Regarding two-round interactive multi-signature schemes, several schemes over pairing-free groups [4,24,33,25] were proposed. However, Drijvers et al. [16] pointed out that these two-round interactive multi-signature schemes [4,24,33,25] are insecure. After the Drijvers et al.'s work, we have some two-round interactive multi-signature schemes on pairing-free groups such as MuSig2 [27] and schemes [1,7,15,35,29,30,34,3].

Ordered Multi-Signatures. Interactive multi-signatures have the drawback that they require interactions among the signers who collaboratively sign a common message to generate a multi-signature. To overcome this drawback, a new primitive called *ordered* multi-signatures was proposed by Boldyreva, Gentry, O'Neill, and Yum [10]. Ordered multi-signatures allow signers to sign a common message sequentially. Similarly to multi-signatures, the size of an ordered multi-signature should be independent of the number of signers. Moreover, ordered multi-signatures allow us to verify the signing order of signers. Ordered multi-signatures have important real-world applications such as network routings [10] and sequential communication delays [6].

Previous Works on Ordered Multi-Signatures. Ordered multi-signature schemes [10,39,38,40,36] were constructed over pairing groups except the recent work by Baum, David, Pagnin, and Takahashi [5]. They proposed the first ordered multi-signature scheme over pairing-free groups. Their scheme is obtained by modifying the recent interactive multi-signature scheme MuSig2 [27]. The security of their ordered multi-signature scheme was proven under the algebraic one-more discrete logarithm (AOMDL) assumption in the random oracle model (ROM).

Ordered Multi-Signatures from the DL Assumption. The AOMDL assumption is weaker than the one-more discrete logarithm (OMDL) assumption but stronger than the discrete logarithm (DL) assumption. To strengthen the assurance of security, it is desirable either to prove their scheme under the DL assumption instead of the AOMDL assumption or propose a construction whose security is proven under the DL assumption.

Current Open Questions. To summarize the above, the following question is unclear in ordered multi-signatures:

Is it possible to prove the security of the Baum et al.'s ordered multi-signature scheme from the DL assumption in the ROM?

1.2 Our Contributions

We provide an affirmative answer to the above question. More precisely, we generalize the Baum et al.'s scheme via a linear hash function and obtain the first DL-based ordered multi-signature scheme. We follow the idea of generalizing of MuSig2 via a linear hash function, as presented in the work by Tessaro and Zhu [35]. They gave a generalized MuSig2, whose security is proven under the AOMDL assumption, by employing a linear hash function. Then, they proved the security of the scheme under the algebraic one-more preimage resistance (AOMPR) property of a linear hash function in the ROM. Similarly, we prove the security of our generalized scheme under the AOMPR property of a linear hash function in the ROM (Theorem 1). Since there is a linear hash function LHF_{Ped} whose AOMPR property is proven under the DL assumption (Lemma 3), we obtain the first ordered multi-signature scheme under the DL assumption in the ROM (Corollary 1).

Comparison between Our Scheme and the Baum et al.'s Scheme. Our scheme $\text{OMS}_{\text{BDPT-H}}[\text{LHF}, \ell]$ generalizes the Baum et al.'s scheme $\text{OMS}_{\text{BDPT}}[\ell]$ via a linear hash function LHF . Both schemes are parameterized by a positive integer ℓ . We recall linear hash functions LHF_{Exp} and LHF_{Ped} over cyclic groups in Fig. 1.

Linear hash function	
$\text{LHF}_{\text{Exp}} : \mathbb{Z}_p \rightarrow \mathbb{G},$	$x \mapsto g^x$
$\text{LHF}_{\text{Ped}} : \mathbb{Z}_p^2 \rightarrow \mathbb{G},$	$(x_1, x_2) \mapsto g_1^{x_1} g_2^{x_2}$

Fig. 1. Two examples of linear hash functions over cyclic groups. p is a prime, $\mathbb{G}$ is a cyclic group of prime order p , and g, g_1 , and g_2 are group element of $\mathbb{G}$. The details of these linear hash functions are given in Section 2.3.

We summarize a comparison between our scheme and the Baum et al.'s scheme in Fig. 2. If we instantiate our scheme $\text{OMS}_{\text{BDPT-H}}[\text{LHF}, \ell]$ with the linear hash function LHF_{Exp} (i.e., $\text{OMS}_{\text{BDPT-H}}[\text{LHF}_{\text{Exp}}, \ell]$), this corresponds to the Baum et al.'s scheme $\text{OMS}_{\text{BDPT}}[\ell]$. If we set $\ell = 2$ and instantiate our scheme $\text{OMS}_{\text{BDPT-H}}[\text{LHF}, \ell]$ with the linear hash function LHF_{Ped} (i.e., $\text{OMS}_{\text{BDPT-H}}[\text{LHF}_{\text{Ped}}, 2]$), we obtain the ordered multi-signature scheme whose security is proven under the DL assumption in the ROM. Compared to $\text{OMS}_{\text{BDPT}}[2]$, our scheme $\text{OMS}_{\text{BDPT-H}}[\text{LHF}_{\text{Ped}}, 2]$ uses the same number of elements in the public key pk . Although our scheme $\text{OMS}_{\text{BDPT-H}}[\text{LHF}_{\text{Ped}}, 2]$ has the weakness that the number of elements in a signature σ slightly increases by an element from $\mathbb{Z}_p$, it has the strength of achieving security under a weaker assumption than the AOMDL assumption that was used for proving the security of $\text{OMS}_{\text{BDPT}}[2]$.

Scheme	Assumption	pk size	σ size
$\text{OMS}_{\text{BDPT}}[\ell = 2]$ Construction 14 in [5]	AOMDL + ROM	$ \mathbb{G} $	$ \mathbb{G} + \mathbb{Z}_p $
$\text{OMS}_{\text{BDPT-H}}[\text{LHF} = \text{LHF}_{\text{Ped}}, \ell = 2]$ Section 4.1	DL + ROM	$ \mathbb{G} $	$ \mathbb{G} + 2 \mathbb{Z}_p $

Fig. 2. Comparison between our scheme and the Baum et al’s scheme in the key registration model. In the column “Scheme”, the $\text{OMS}_{\text{BDPT}}[\ell]$ represents the proposed scheme in [5] that is parameterized by ℓ . The scheme $\text{OMS}_{\text{BDPT-H}}[\text{LHF}, \ell]$ is our generalized scheme of $\text{OMS}_{\text{BDPT-H}}$ with a linear hash function LHF. Linear hash functions LHF_{Exp} and LHF_{Ped} are cyclic group-based linear hash functions given Fig. 1. The column “pk size” represents the size of a public key pk. The column “ σ size” represents the size of a signature σ . $|\mathbb{G}|$ represents the size of a group element in $\mathbb{G}$ and $|\mathbb{Z}_p|$ represents the size of a group element in $\mathbb{Z}_p$ where p is a prime number.

1.3 More Related Works

In this section, we review related works for ordered multi-signatures.

Sequential Aggregate Signatures and Ordered Multi-Signatures. A concept similar to ordered multi-signatures is sequential aggregate signatures [23]. Sequential aggregate signatures are a variant of aggregate signatures [12], in which different signers sequentially sign distinct messages to produce an aggregated signature. The size of a generated signature should be independent of the number of signers. We have a lot of sequential aggregate signature schemes over pairing groups (e.g., [23, 8, 21, 10, 17, 20, 22, 32]).

Ordered multi-signatures [10] allow signers to sign a *common* message sequentially. A generated signature has a property that the signing order of signers is verifiable. Ordered multi-signatures can be regarded as a restricted variant of sequential aggregate signatures. In fact, Boldyreva et al. [10] explained that ordered multi-signatures can be obtained from sequential aggregate signatures by forcing the following modifications to signers. The i -th signer signs (m, i) instead of signing a distinct message m_i where m is the common message for all signers. Due to this fact, research on a direct construction of ordered multi-signatures only makes sense if it offers a strength over obvious schemes derived from sequential aggregate signatures.

The trivial way to obtain a pairing-free ordered multi-signature scheme is to apply the Boldyreva et al.’s modifications to an existing pairing-free sequential aggregate signature scheme by Chen and Zhao [14]. However, the resulting ordered multi-signature scheme from their sequential aggregate signature scheme has a drawback: the signature size grows linearly with the number of signers. By contrast, the ordered multi-signature scheme by Baum et al. [5] and our proposed scheme offer compact signatures, meaning that the signature size is constant with respect to the number of signers.

Security Models for Ordered Multi-Signatures. The game-based security of ordered multi-signatures was proposed in [10,39]. Many previous works [10,39,38,40,36] proposed ordered multi-signature schemes and proved that these schemes satisfy the game-based security in the key registration model. Baum et al. [5] revisited the security model, proposed the UC-based security model and a new game-based security model, and proved that their game-based security implies the UC-based security. In this work, we consider their game-based security model in the key registration model (Definition 7).

1.4 Paper Organization

In Section 2, first, we introduce notations and the forking lemma. Then, we review linear hash functions and these instantiations over cyclic groups. In Section 3, we review the definition of an ordered multi-signature scheme and its security. In Section 4, we give our new scheme $\text{OMS}_{\text{BDPT-H}}[\text{LHF}, \ell]$ by generalizing the $\text{OMS}_{\text{BDPT}}[\ell]$ with a linear hash function LHF. Then, we prove the security of our scheme. In Appendix A, we review the OMDL assumption and its variant. In Appendix B, we give the Baum et al.'s scheme $\text{OMS}_{\text{BDPT}}[\ell]$.

2 Preliminaries

In this section, first, we introduce notations and recall the forking lemma, which is used for the security analysis of our scheme. Then, we recall a linear hash function and its instantiations LHF_{Exp} and LHF_{Ped} over cyclic groups.

2.1 Notations

Let λ be the security parameter. A function $f(\lambda)$ is negligible in λ if $f(\lambda)$ tends to 0 faster than $\frac{1}{\lambda^c}$ for every constant $c > 0$. Let $\text{negl}(\lambda)$ denote a negligible function in λ and $\text{poly}(\lambda)$ denote a polynomial function in λ . For a positive integer n , we define $[n] := \{1, \dots, n\}$. For a finite set S , $s \xleftarrow{\$} S$ represents that an element s is chosen from S uniformly at random, and $|S|$ represents the number of elements in S . We denote an infinite set of all finite bit strings as $\{0, 1\}^*$. For an algorithm A , $y \leftarrow A(x)$ denotes that the algorithm A outputs y on input x . We abbreviate probabilistic polynomial time as PPT.

For a cyclic group $\mathbb{G}$, we define $\mathbb{G}^* := \mathbb{G} \setminus \{1_{\mathbb{G}}\}$ where $1_{\mathbb{G}}$ is the identity element of $\mathbb{G}$. We introduce a group generator GrGen . It takes a security parameter 1^λ and outputs a description $(\mathbb{G}, p, g)$ of a cyclic group where $\mathbb{G}$ is a cyclic group of a λ -bits prime order p and $g \in \mathbb{G}^*$ is a generator of $\mathbb{G}$. We review the DL assumption.

Definition 1 (DL assumption). Let GrGen be a cyclic group generator and A be a PPT adversary. The discrete logarithm (DL) assumption on GrGen is defined by the DL game $\mathcal{G}_{\text{GrGen}, A}^{\text{DL}}$ in Fig. 3. The advantage of A is defined as $\text{Adv}_{\text{GrGen}, A}^{\text{DL}}(\lambda) := \Pr[\mathcal{G}_{\text{GrGen}, A}^{\text{DL}}(1^\lambda) \Rightarrow 1]$. The DL assumption holds on GrGen if for any PPT adversary A , $\text{Adv}_{\text{GrGen}, A}^{\text{DL}}(\lambda)$ is $\text{negl}(\lambda)$.

GAME $G_{\text{GrGen}, A}^{\text{DL}}(1^\lambda)$:

$\text{par} = (\mathbb{G}, p, g) \leftarrow \text{GrGen}(1^\lambda)$, $x \xleftarrow{\$} \mathbb{Z}_p$, $X \leftarrow g^x$, $x' \leftarrow A(\text{par}, X)$.

If $x = x'$, return 1.

Otherwise, return 0.

Fig. 3. The DL game $G_{\text{GrGen}, A}^{\text{DL}}$.

2.2 Generalized Forking Lemma

We recall the generalized forking lemma [28,5].

Lemma 1 (Generalized forking lemma [28,5]). *Fix integers q and m and sets C and V such that $|C| > 2$ and $|V| > 2$. Let IGen be a randomized input generator algorithm that takes a security parameter 1^λ as input, and outputs a main input inp . Let B be a randomized algorithm that takes a main input inp , elements $c_1, \dots, c_q \in C$, and $v_1, \dots, v_m \in V$ as input, and outputs two indices $i \in [0, \dots, q]$ and $j \in [0, \dots, m]$ and a side output out . The forking algorithm Fork^B associated to B is defined in Fig. 4. Let acc and frk be the*

$\text{Fork}^B(\text{inp}, v_1, \hat{v}_1, \dots, v_m, \hat{v}_m)$:

Choose a random tape ρ of B , $(c_1, \dots, c_q) \xleftarrow{\$} C$,

$(i, j, \text{out}) \leftarrow B(\text{inp}, (c_1, \dots, c_q), (v_1, \dots, v_m); \rho)$.

If $i = 0$, return $(0, \perp, \perp)$.

$(\hat{c}_1, \dots, \hat{c}_q) \xleftarrow{\$} C$,

$(\hat{i}, \hat{j}, \hat{\text{out}}) \leftarrow B(\text{inp}, (c_1, \dots, c_{i-1}, \hat{c}_i, \dots, \hat{c}_q), (v_1, \dots, v_j, \hat{v}_{j+1}, \dots, \hat{v}_m); \rho)$.

If $i = \hat{i} \wedge c_i \neq \hat{c}_i$, return $(1, \text{out}, \hat{\text{out}})$.

Otherwise, return $(0, \perp, \perp)$.

Fig. 4. The forking algorithm Fork^B associated to B .

following probabilities:

$$\begin{aligned} \text{acc} &:= \Pr \left[i \geq 1 : \text{inp} \leftarrow \text{IGen}(1^\lambda), (c_1, \dots, c_q) \xleftarrow{\$} C, (v_1, \dots, v_m) \xleftarrow{\$} V^m \right. \\ &\quad \left. (i, j, \text{out}) \leftarrow B(\text{inp}, (c_1, \dots, c_q), (v_1, \dots, v_m)) \right], \\ \text{frk} &:= \Pr \left[b = 1 : \text{inp} \leftarrow \text{IGen}(1^\lambda), (v_1, \hat{v}_1, \dots, v_m, \hat{v}_m) \xleftarrow{\$} V^{2m}, \right. \\ &\quad \left. (b, \text{out}, \hat{\text{out}}) \leftarrow \text{Fork}^B(\text{inp}, v_1, \hat{v}_1, \dots, v_m, \hat{v}_m) \right]. \end{aligned}$$

Then, the following holds:

$$\text{frk} \geq \text{acc} \cdot \left(\frac{\text{acc}}{Q} - \frac{1}{|C|} \right) \quad \text{alternatively,} \quad \text{acc} \leq \frac{Q}{|C|} + \sqrt{Q \cdot \text{frk}}.$$

2.3 Linear Hash Function

Before defining a linear hash function, first, we give the definition of modules.

Modules. Let $\mathcal{S}$ be a ring with a multiplicative identity element $1_{\mathcal{S}}$ and $\mathcal{M}$ be an additive abelian group with an identity element $0_{\mathcal{M}}$. We say that $\mathcal{M}$ is an $\mathcal{S}$ -module if there exists an operation $\cdot : \mathcal{S} \times \mathcal{M} \rightarrow \mathcal{M}$ such that for all $r, s \in \mathcal{S}$ and $x, y \in \mathcal{M}$, (1) $r \cdot (x + y) = r \cdot x + r \cdot y$, (2) $(a + b) \cdot x = a \cdot x + b \cdot x$, (3) $(ab) \cdot x = a \cdot (b \cdot x)$, and (4) $1_{\mathcal{S}} \cdot x = x$ hold.

Linear Hash Function. We review the definition of the linear hash function.

Definition 2 (Linear hash function). A linear hash function LHF is a pair of algorithms (PGen, F) defined as follows.

- $\text{PGen}(1^\lambda)$: The parameter generation algorithm PGen is a randomized algorithm that takes a security parameter 1^λ as input, and outputs a public parameter par . The public parameter par defines three sets $\mathcal{S} = \mathcal{S}(\text{par})$, $\mathcal{D} = \mathcal{D}(\text{par})$, and $\mathcal{R} = \mathcal{R}(\text{par})$ where $\mathcal{S}$ is a field and $\mathcal{D}$ and $\mathcal{R}$ are $\mathcal{S}$ -modules.
- $\text{F}(\text{par}, x)$: The evaluation algorithm F is a deterministic algorithm that takes a public parameter par and an element $x \in \mathcal{D}$ as input, and outputs an element $y \in \mathcal{R}$. $\text{F}(\text{par}, \cdot) : \mathcal{D} \rightarrow \mathcal{R}$ is required to be a homomorphism of $\mathcal{S}$ -modules (i.e., for any $r \in \mathcal{R}$, $x, x' \in \mathcal{D}$, $\text{F}(\text{par}, x + r \cdot x') = \text{F}(\text{par}, x) + r\text{F}(\text{par}, x')$ holds.)

Definition 3 (Collision resistance). Let $\text{LHF} = (\text{PGen}, \text{F})$ be a linear function and $\mathbf{A}$ be a PPT adversary. The collision resistance property of LHF is defined by the collision resistance game $\text{G}_{\text{LHF}, \mathbf{A}}^{\text{CR}}$ in Fig. 5. The advantage of $\mathbf{A}$ is defined as $\text{Adv}_{\text{LHF}, \mathbf{A}}^{\text{CR}}(\lambda) := \Pr[\text{G}_{\text{LHF}, \mathbf{A}}^{\text{CR}}(1^\lambda) \Rightarrow 1]$. LHF satisfies the collision resistance property if for any PPT adversary $\mathbf{A}$, $\text{Adv}_{\text{LHF}, \mathbf{A}}^{\text{CR}}(\lambda)$ is $\text{negl}(\lambda)$.

GAME $\text{G}_{\text{LHF}, \mathbf{A}}^{\text{CR}}(1^\lambda)$:

$\text{par} \leftarrow \text{LHF}(1^\lambda)$, $(x, x') \leftarrow \mathbf{A}(\text{par})$.

If $\text{F}(\text{par}, x) = \text{F}(\text{par}, x') \wedge x \neq x'$, return 1.

Otherwise, return 0.

Fig. 5. The collision resistance game for LHF $\text{G}_{\text{LHF}, \mathbf{A}}^{\text{CR}}$.

Definition 4 (Algebraic one-more preimage resistance [35]). Let $\text{LHF} = (\text{PGen}, \text{F})$ be a linear function and $\mathbf{A}$ be a PPT adversary. The algebraic one-more preimage resistance (AOMPR) property of LHF is defined by the algebraic one-more preimage resistance game $\text{G}_{\text{LHF}, \mathbf{A}}^{\text{AOMPR}}$ in Fig. 6. The advantage of $\mathbf{A}$ is defined as $\text{Adv}_{\text{LHF}, \mathbf{A}}^{\text{AOMPR}}(\lambda) := \Pr[\text{G}_{\text{LHF}, \mathbf{A}}^{\text{AOMPR}}(1^\lambda) \Rightarrow 1]$. LHF satisfies the AOMPR property if for any PPT adversary $\mathbf{A}$, $\text{Adv}_{\text{LHF}, \mathbf{A}}^{\text{AOMPR}}(\lambda)$ is $\text{negl}(\lambda)$.

GAME $G_{\text{LHF},A}^{\text{AOMPR}}(1^\lambda)$: $\text{par} \leftarrow \text{LHF}(1^\lambda)$, $q_{\text{CH}} \leftarrow 0$, $q_{\text{AInv}} \leftarrow 0$, $x_0 \xleftarrow{\$} \mathcal{D}(\text{par})$, $X_0 \leftarrow \text{F}(\text{par}, x_0)$, $(x'_0, \dots, x'_{q_{\text{CH}}}) \leftarrow \mathcal{A}^{\mathcal{O}^{\text{CH}}(), \mathcal{O}^{\text{AInv}}(\cdot, \cdot)}(\text{par}, X_0)$. If $(\text{F}(\text{par}, x'_0), \dots, \text{F}(\text{par}, x'_{q_{\text{CH}}})) = (X_0, \dots, X_{q_{\text{CH}}}) \wedge q_{\text{CH}} \geq q_{\text{AInv}}$, return 1. Otherwise, return 0.
Oracle $\mathcal{O}^{\text{CH}}()$: $q_{\text{CH}} \leftarrow q_{\text{CH}} + 1$, $x_{q_{\text{CH}}} \xleftarrow{\$} \mathcal{D}(\text{par})$, $X_{q_{\text{CH}}} \leftarrow \text{F}(\text{par}, x_{q_{\text{CH}}})$, return $X_{q_{\text{CH}}}$.
Oracle $\mathcal{O}^{\text{AInv}}(X, (\alpha, (\beta_i)_{i \in \{0, \dots, q_{\text{CH}}\}}))$: If $X \neq \text{F}(\text{par}, \alpha) + \sum_{i \in \{0, \dots, q_{\text{CH}}\}} \beta_i X_i$, return $\perp$. $q_{\text{AInv}} \leftarrow q_{\text{AInv}} + 1$, $x \leftarrow \alpha + \sum_{i \in [\text{cid}]} \beta_i x_i$, return z .

Fig. 6. The algebraic one-more preimage resistance game for LHF $G_{\text{LHF},A}^{\text{AOMPR}}$.

Lemma 2 ([35]). *If a linear hash function LHF satisfies the collision resistance property, then LHF satisfies the AOMPR property.*

As the instantiations of LHF, we focus on the following linear hash functions LHF_{Exp} and LHF_{Ped} from cyclic groups. Let GrGen be a cyclic group generator.

Exponentiation Linear Hash Function LHF_{Exp} . The exponentiation linear hash function $\text{LHF}_{\text{Exp}}[\text{GrGen}] = (\text{PGen}, \text{F})$ is given as follows.

- $\text{PGen}(1^\lambda)$: Return $\text{par} = (\mathbb{G}, p, g) \leftarrow \text{GrGen}(1^\lambda)$.
 The public parameter par defines $\mathcal{S}(\text{par}) := \mathbb{Z}_p$, $\mathcal{D}(\text{par}) = \mathbb{Z}_p$, $\mathcal{R}(\text{par}) = \mathbb{G}$.
- $\text{F}(\text{par} = (\mathbb{G}, p, g), x)$: Return g^x .

Later, LHF_{Exp} is used to describe the ordered multi-signature scheme OMS_{BDPT} [5] in Section 4. It is clear that the AOMPR property of LHF_{Exp} is implied by the AOMDL assumption.

Pedersen Linear Hash Function LHF_{Ped} [31]. The Pedersen linear hash function $\text{LHF}_{\text{Ped}}[\text{GrGen}] = (\text{PGen}, \text{F})$ is given as follows.

- $\text{PGen}(1^\lambda)$: $\text{par} = (\mathbb{G}, p, g_1) \leftarrow \text{GrGen}(1^\lambda)$, $g_2 \xleftarrow{\$} \mathbb{G}^*$, return $\text{par} = (\mathbb{G}, p, g_1, g_2)$.
 The public parameter par defines $\mathcal{S}(\text{par}) := \mathbb{Z}_p$, $\mathcal{D}(\text{par}) = \mathbb{Z}_p^2$, $\mathcal{R}(\text{par}) = \mathbb{G}$.
 The operation over $\mathcal{D}(\text{par})$ is defined as follows. For any $(x_1, x_2), (x'_1, x'_2) \in \mathcal{D}(\text{par})$ and $s \in \mathcal{S}(\text{par})$, $(x_1, x_2) + (x'_1, x'_2) = (x_1 + x'_1, x_2 + x'_2)$ and $s \cdot (x_1, x_2) = (sx_1, sx_2)$.
- $\text{F}(\text{par} = (\mathbb{G}, p, g_1, g_2), (x_1, x_2))$: Return $g_1^{x_1} g_2^{x_2}$.

For the Pedersen linear hash function LHF_{Ped} , the following fact is known.

Lemma 3 ([35]). *If the DL assumption holds on GrGen , LHF_{Ped} satisfies the collision resistance property.*

From Lemma 2 and Lemma 3, the AOMPR property of LHF_{Ped} is implied by the DL assumption.

3 Ordered Multi-Signatures

In this section, we review the definition of an ordered multi-signature scheme and recall the Baum et al.'s ordered multi-signature scheme $\text{OMS}_{\text{BDPT}}[\ell]$ [5].

3.1 Syntax

Here, we review the definition of an ordered multi-signature scheme and its correctness and unforgeability.

Definition 5 (Ordered multi-signatures [5]). *An ordered multi-signature (OMS) scheme OMS consists of a tuple of algorithms $(\text{Setup}, \text{KeyGen}, \text{SignOff}, \text{SignOn}, \text{Verify})$ defined as follows.*

- $\text{Setup}(1^\lambda)$: *The setup algorithm takes a security parameter 1^λ as input, and outputs a public parameter pp which includes an initial aggregate σ_0 so far as a public constant. We write this element as $\text{pp}.\sigma_0$.*
- $\text{KeyGen}(\text{pp})$: *The key generation algorithm takes a public parameter pp as input, and outputs a public key pk and a secret key sk .*
- $\text{SignOff}(\text{pp}, \text{sk})$: *The offline signing algorithm takes a public parameter pp and a secret key sk as input, and outputs an offline token off and an internal state st .*
- $\text{SignOn}(\text{pp}, \text{sk}, \text{st}, L, m, \text{offs}, \sigma)$: *The online signing algorithm takes a public parameter pp , a secret key sk , an internal state st , an ordered list of public keys $L = (\text{pk}_1, \dots, \text{pk}_n)$, a message m , some offline-token information $\text{offs} = (\text{off}_1, \dots, \text{off}_n)$, and a so-far-aggregated online signature σ as input. It outputs a tuple (σ', b') where σ' is a new so-far-aggregated online signature and $b' \in \{0, 1\}$ is a bit that indicates the validity of σ' .*
- $\text{Verify}(\text{pp}, L, m, \sigma)$: *The verification algorithm takes a public parameter pp , an ordered list of public keys $L = (\text{pk}_1, \dots, \text{pk}_n)$, a message m , and a signature σ as input, and outputs a bit $b \in \{0, 1\}$.*

Definition 6 (Correctness [5]). *Let $\text{OMS} = (\text{Setup}, \text{KeyGen}, \text{SignOff}, \text{SignOn}, \text{Verify})$ be an ordered multi-signature scheme and $\mathcal{A}$ be an adversary. The correctness of OMS is defined by the correctness game $\text{G}_{\text{OMS}, \mathcal{A}}^{\text{Correct}}$ in Fig. 7. We say that OMS satisfies correctness if for any PPT adversary $\mathcal{A}$, $\Pr[\text{G}_{\text{OMS}, \mathcal{A}}^{\text{Correct}}(1^\lambda) \Rightarrow 1]$ is $\text{negl}(\lambda)$.*

3.2 Security

Definition 7 (Unforgeability in the key registration model [5]). *Let $\text{OMS} = (\text{Setup}, \text{KeyGen}, \text{SignOff}, \text{SignOn}, \text{Verify})$ be an ordered multi-signature scheme and $\mathcal{A}$ be an adversary. The existentially unforgeable under chosen message attacks in the key registration model (EUF-CMA-KRM) security of OMS is defined by the unforgeability game $\text{G}_{\text{OMS}, \mathcal{A}}^{\text{EUF-CMA-KRM}}$ in Fig. 8. The advantage of $\mathcal{A}$ is defined as $\text{Adv}_{\text{OMS}, \mathcal{A}}^{\text{EUF-CMA-KRM}}(\lambda) := \Pr[\text{G}_{\text{OMS}, \mathcal{A}}^{\text{EUF-CMA-KRM}}(1^\lambda) \Rightarrow 1]$. We say that OMS satisfies the EUF-CMA-KRM security if for any PPT adversary $\mathcal{A}$, $\text{Adv}_{\text{OMS}, \mathcal{A}}^{\text{EUF-CMA-KRM}}(\lambda)$ is $\text{negl}(\lambda)$.*

GAME $G_{\text{OMS},A}^{\text{Correct}}(1^\lambda)$: $\text{pp} \leftarrow \text{Setup}(1^\lambda), \mathcal{K} \leftarrow \{\}, \mathcal{T} \leftarrow \{\}, \mathcal{Q} \leftarrow \{\}, \text{win} \leftarrow 0.$ $(L, m, \text{offs}, \sigma) \leftarrow A^{\mathcal{O}^{\text{KeyReg}}(\cdot, \cdot), \mathcal{O}^{\text{SignOff}}(\cdot, \cdot), \mathcal{O}^{\text{SignOn}}(\cdot, \cdot, \cdot, \cdot, \cdot, \cdot)}(\text{pp})$ $(\text{pk}_1, \dots, \text{pk}_n) \leftarrow L, (\text{off}_1, \dots, \text{off}_n) \leftarrow \text{offs}, \sigma_0 \leftarrow \text{pp}.\sigma_0, \sigma_n \leftarrow \sigma.$ If $\forall j \in [n], \exists \text{uid}_j \text{ s.t. } \mathcal{K}[\text{uid}_j] = (1, \text{pk}, \cdot)$ $\wedge \forall j \in [n], \exists \text{uid}_j, \text{ssid}_j \text{ s.t. } \mathcal{T}[\text{ssid}_j, \text{uid}_j] = (\cdot, \text{off}_i)$ $\wedge \exists (\sigma_1, \dots, \sigma_{i-1}), \forall j \in [i-1] \text{ s.t. } (L, m, \text{offs}, \sigma_{j-1}, \sigma_j, b_{j-1}, j) \in \mathcal{Q}$ $\wedge \text{Verify}(\text{pp}, L, m, \sigma) = 0$, then $\text{win} \leftarrow 1.$ Return win.
Oracle $\mathcal{O}^{\text{KeyReg}}(\text{uid}, \text{aux})$: If $\mathcal{K}[\text{uid}] \neq \perp$, then return $\perp.$ If $\text{aux} = \epsilon$, then $(\text{pk}, \text{sk}) \leftarrow \text{KeyGen}(\text{pp}), \mathcal{K}[\text{uid}] \leftarrow (1, \text{pk}, \text{sk}).$ If $\text{aux} \neq \epsilon$, then $(\text{pk}, \text{sk}, \rho) \leftarrow \text{aux},$ If $(\text{pk}, \text{sk}) \neq \text{KeyGen}(\text{pp}; \rho)$, then return $\perp.$ Otherwise, $\mathcal{K}[\text{uid}] \leftarrow (0, \text{pk}, \text{sk}).$ Return pk.
Oracle $\mathcal{O}^{\text{SignOff}}(\text{ssid}, \text{uid})$: $(b, \text{pk}, \text{sk}) \leftarrow \mathcal{K}[\text{uid}].$ If $(b = 0 \vee \text{sk} = \epsilon) \vee (\mathcal{T}[\text{ssid}, \text{uid}] \neq \perp)$, then return $\perp.$ $(\text{off}, \text{st}) \leftarrow \text{SignOff}(\text{pp}, \text{sk}), \mathcal{T}[\text{ssid}, \text{uid}] \leftarrow (\text{off}, \text{st}).$ Return off.
Oracle $\mathcal{O}^{\text{SignOn}}(\text{ssid}, \text{uid}, L, m, \text{offs}, \sigma_{i-1}, i)$: $(b, \text{pk}, \text{sk}) \leftarrow \mathcal{K}[\text{uid}], (\text{pk}_1, \dots, \text{pk}_n) \leftarrow L, (\text{off}_1, \dots, \text{off}_n) \leftarrow \text{offs}.$ If $b = 0 \vee \text{sk} = \epsilon$, then return $\perp.$ If $\mathcal{T}[\text{ssid}, \text{uid}] \neq (\cdot, \text{off}_i) \vee \mathcal{Q}[\text{ssid}, \text{uid}] \neq \perp$, then return $\perp.$ If $(\text{pk} \neq \text{pk}_i) \vee (i = 1 \wedge \sigma_{i-1} \neq \text{pp}.\sigma_0)$, then return $\perp.$ If $\exists j \in [n] \text{ s.t. } (*, \text{pk}_j, *) \notin \mathcal{K}$, then return $\perp.$ $(\text{st}_i, \text{off}_i) \leftarrow \mathcal{T}[\text{ssid}, \text{uid}], (\sigma_i, b_{i-1}) \leftarrow \text{SignOn}(\text{pp}, \text{sk}, \text{st}_i, L, m, \text{offs}, \sigma_{i-1}),$ $\mathcal{Q}[\text{ssid}, \text{uid}] \leftarrow (L, m, \text{offs}, \sigma_{i-1}, \sigma_i, b_{i-1}, i)$ If $\forall j \in [n], \exists \text{uid}_j \text{ s.t. } \mathcal{K}[\text{uid}_j] = (1, \text{pk}, \cdot)$ $\wedge \forall j \in [n], \exists \text{uid}_j, \exists \text{ssid}_j \text{ s.t. } \mathcal{T}[\text{ssid}_j, \text{uid}_j] = (\cdot, \text{off}_i)$ $\wedge \exists (\sigma_1, \dots, \sigma_{i-1}), \forall j \in [i-1] \text{ s.t. } (L, m, \text{offs}, \sigma_{j-1}, \sigma_j, b_{j-1}, j) \in \mathcal{Q}$ $\wedge b_{i-1} = 0$, then $\text{win} \leftarrow 1.$ Return $(\sigma_i, b_{i-1}).$

Fig. 7. The correctness game for OMS $G_{\text{OMS},A}^{\text{Correct}}$.

GAME $\mathbf{G}_{\text{OMS},A}^{\text{EUF-CMA-KRM}}(1^\lambda)$: $\text{pp} \leftarrow \text{Setup}(1^\lambda)$, $\mathcal{K} \leftarrow \{\}$, $\mathcal{T} \leftarrow \{\}$, $\mathcal{Q} \leftarrow \{\}$, $\text{win} \leftarrow 0$, $(L^*, m^*, \sigma^*) \leftarrow \mathcal{A}^{\mathcal{O}^{\text{KeyReg}}(\cdot, \cdot), \mathcal{O}^{\text{SignOff}}(\cdot, \cdot), \mathcal{O}^{\text{SignOn}}(\cdot, \cdot, \cdot, \cdot, \cdot, \cdot)}(\text{pp})$. If $\text{win} = 1$, then return 1. $(\text{pk}_1, \dots, \text{pk}_n) \leftarrow L^*$. If $\exists \text{pk} \in L^*$ s.t. $(\cdot, \text{pk}, \cdot) \notin \mathcal{K}$, then return 0. If $\text{Verify}(\text{pp}, L^*, m^*, \sigma^*) = 0$, then return 0. If $\exists j \in [n]$ s.t. $(1, \text{pk}_j, \cdot) \in \mathcal{K} \wedge (L^*, m^*, j) \notin \mathcal{Q}$, then return 1. Otherwise, return 0.
Oracle $\mathcal{O}^{\text{KeyReg}}(\text{uid}, \text{aux})$: If $\mathcal{K}[\text{uid}] \neq \perp$, then return $\perp$. If $\text{aux} = \epsilon$, then $(\text{pk}, \text{sk}) \leftarrow \text{KeyGen}(\text{pp})$, $\mathcal{K}[\text{uid}] \leftarrow (1, \text{pk}, \text{sk})$. If $\text{aux} \neq \epsilon$, then $(\text{pk}, \text{sk}, \rho) \leftarrow \text{aux}$, If $(\text{pk}, \text{sk}) \neq \text{KeyGen}(\text{pp}; \rho)$, return $\perp$. Otherwise, $\mathcal{K}[\text{uid}] \leftarrow (0, \text{pk}, \text{sk})$. Return pk.
Oracle $\mathcal{O}^{\text{SignOff}}(\text{ssid}, \text{uid})$: $(b, \text{pk}, \text{sk}) \leftarrow \mathcal{K}[\text{uid}]$. If $(b = 0 \vee \text{sk} = \epsilon) \vee (\mathcal{T}[\text{ssid}, \text{uid}] \neq \perp)$, return $\perp$. $(\text{off}, \text{st}) \leftarrow \text{SignOff}(\text{pp}, \text{sk})$, $\mathcal{T}[\text{ssid}, \text{uid}] \leftarrow (\text{off}, \text{st})$. Return off.
Oracle $\mathcal{O}^{\text{SignOn}}(\text{ssid}, \text{uid}, L, m, \text{offs}, \sigma_{i-1}, i)$: $(b, \text{pk}, \text{sk}) \leftarrow \mathcal{K}[\text{uid}]$, $(\text{pk}_1, \dots, \text{pk}_n) \leftarrow L$, $(\text{off}_1, \dots, \text{off}_n) \leftarrow \text{offs}$. If $b = 0 \vee \text{sk} = \epsilon$, return $\perp$. If $\mathcal{T}[\text{ssid}, \text{uid}] \neq (\text{off}_i, \cdot) \vee \mathcal{Q}[\text{ssid}, \text{uid}] \neq \perp$, return $\perp$. If $(\text{pk} \neq \text{pk}_i) \vee (i = 1 \wedge \sigma_{i-1} \neq \text{pp}.\sigma_0)$, return $\perp$. If $\exists j \in [n]$ s.t. $(\cdot, \text{pk}_j, \cdot) \notin \mathcal{K}$, return $\perp$. $(\text{st}_i, \text{off}_i) \leftarrow \mathcal{T}[\text{ssid}, \text{uid}]$, $(\sigma_i, b_{i-1}) \leftarrow \text{SignOn}(\text{pp}, \text{sk}, \text{st}_i, L, m, \text{offs}, \sigma_{i-1})$, $\mathcal{Q}[\text{ssid}, \text{uid}] \leftarrow (L, m, i)$. If $b_{i-1} = 1 \wedge (\exists j \in [i-1] \text{ s.t. } ((1, \text{pk}_j, \cdot) \in \mathcal{K} \wedge (L, m, j) \notin \mathcal{Q}))$, $\text{win} \leftarrow 1$. Return (σ_i, b_{i-1}).

Fig. 8. The EUF-CMA-KRM security game for OMS $\mathbf{G}_{\text{OMS},A}^{\text{EUF-CMA-KRM}}$.

4 Ordered Multi-Signatures from the DL Assumption

In this section, we first give the description of our ordered multi-signature scheme from a linear hash function. Then, we prove the security of our scheme.

4.1 Description

We generalize the ordered multi-signature scheme $\text{OMS}_{\text{BDPT}}[\ell]$ in [5] using a linear hash function. Let $\text{LHF} = (\text{PGen}, \text{F})$ be a linear hash function with an $\mathcal{S}$ -modules domain and an $\mathcal{S}$ -modules range $\mathcal{R}$. The description of our ordered multi-signature scheme $\text{OMS}_{\text{BDPT}}[\text{LHF}, \ell]$ is given in Fig. 9. Note that the scheme $\text{OMS}_{\text{BDPT-H}}[\text{LHF}, \ell]$ with $\text{LHF} = \text{LHF}_{\text{Exp}}$ corresponds to $\text{OMS}_{\text{BDPT}}[\ell]$. The correctness analysis of $\text{OMS}_{\text{BDPT-H}}[\text{LHF}, \ell]$ follows that of $\text{OMS}_{\text{BDPT}}[\ell]$ in [5].

4.2 Security Analysis

We give the security analysis for our OMS scheme $\text{OMS}_{\text{BDPT-H}}[\text{LHF}, \ell]$.

Theorem 1. *If LHF satisfies the AOMPR property, $\text{OMS}_{\text{BDPT-H}}[\text{LHF}, 2]$ satisfies the EUF-CMA-KRM security in the ROM.*

Proof. We prove the theorem by considering the following two types of adversaries $\mathcal{A}$.

- Type 1: $\mathcal{A}$ outputs (L^*, m^*, σ^*) at the end of the game.
- Type 2: $\mathcal{A}$ makes a query $(L^*, m^*, \text{offs}^*, \sigma_{i^*-1}^*, i^*)$ to $\mathcal{O}^{\text{SignOn}}$, and the flag win is changed to 1.

Reduction against the Type 1 Adversary. Let $\mathcal{A}$ be a type 1 adversary for the EUF-CMA-KRM security game $\mathcal{G}_{\text{OMS}_{\text{BDPT}}[\text{LHF}, 2], \mathcal{A}}^{\text{EUF-CMA-KRM}}$ that receives at most Q_k public keys from $\mathcal{O}^{\text{KeyReg}}$, makes at most Q_h queries to the random oracles, starts at most Q_s sessions in total with $\mathcal{O}^{\text{SignOff}}$, and outputs $(L^*, m^*, \sigma^* = (R^*, z^*))$ at the end of the game. Let $Q := Q_s + 2Q_h + 1$.

We construct the reduction $\mathcal{R}_1$ by the following two steps. In the first step, we construct an algorithm $\mathcal{B}_1^{\mathcal{O}^{\text{CH}}, \mathcal{O}^{\text{Alnv}}, \mathcal{A}(\cdot)}$ which simulates the view of $\mathcal{A}$ without using a secret key belonging to one of uncorrupted parties, where $\mathcal{O}^{\text{CH}}$ and $\mathcal{O}^{\text{Alnv}}$ are oracles defined in the same way as those in the algebraic one-more preimage resistance game in Definition 4. In the second step, we use $\mathcal{B}_1$ to construct the reduction $\mathcal{R}_1$ which runs the forking algorithm $\text{Fork}^{\mathcal{B}_1}$ and extracts the preimage of LHF. Now, we give the detail of $\mathcal{B}_1$ and $\mathcal{R}_1$.

The Algorithm $\mathcal{B}_1^{\mathcal{O}^{\text{CH}}, \mathcal{O}^{\text{Alnv}}, \mathcal{A}}$ ($\text{par}, v_1, \dots, v_Q, c_1, \dots, c_Q$). The algorithm $\mathcal{B}_1$ takes a tuple $(\text{par}, v_1, \dots, v_Q, c_1, \dots, c_Q)$ as input, where par is a public parameter of LHF and $(v_1, \dots, v_Q, c_1, \dots, c_Q) \in \mathcal{S}(\text{par})^{2Q}$. $\mathcal{B}_1$ simulates the game $\mathcal{G}_{\text{LHF}, \mathcal{A}}^{\text{AOMPR}}$ to $\mathcal{A}$ by accessing oracles $\mathcal{O}^{\text{CH}}$ and $\mathcal{O}^{\text{Alnv}}$ that are defined in the same way as the game $\mathcal{G}_{\text{LHF}, \mathcal{A}}^{\text{AOMPR}}$. For simplicity of discussion, when $\mathcal{B}_1$ makes a query $(Y, \alpha, (\beta_i)_{i \in [\text{cid}]})$, we omit the coefficient $\alpha, (\beta_i)_{i \in [\text{cid}]}$ whenever they are clear from the context. After receiving the input, $\mathcal{B}_1$ proceeds as follows:

Setup(1^λ) : $\text{par} \leftarrow \text{LHF}(1^\lambda)$, $n = \text{poly}(\lambda)$, $\sigma_0 = (0_{\mathcal{R}}, 1_{\mathcal{S}})$. Choose hash functions $H_{\text{non}}, H_{\text{ch}} : \{0, 1\}^* \rightarrow \mathcal{S}(\text{par})$. Return $\text{pp} \leftarrow (\text{par}, n, H_{\text{non}}, H_{\text{ch}}, \sigma_0)$.
KeyGen(pp) : $\text{sk} \xleftarrow{\\$} \mathcal{D}$, $\text{pk} \leftarrow \text{F}(\text{par}, \text{sk})$. Return (pk, sk).
SignOff(pp, sk_i) : For $j \in [\ell]$, $r_{i,j} \xleftarrow{\\$} \mathcal{D}$, $R_{i,j} \leftarrow \text{F}(\text{par}, r_{i,j})$. Return $(\text{off}_i, \text{st}_i) \leftarrow ((R_{i,1}, \dots, R_{i,\ell}), (r_{i,1}, \dots, r_{i,\ell}))$.
SignOn($\text{pp}, \text{sk}, \text{st}_i, L, m, \text{offs} = (\text{off}_k)_{k \in [n]}, \sigma_{i-1}$) : $(\text{pk}_1, \dots, \text{pk}_n) \leftarrow L$, $(r_{i,1}, \dots, r_{i,\ell}) \leftarrow \text{st}_i$, $((R_{k,1}, \dots, R_{k,\ell})_{k \in [n]}) \leftarrow \text{offs}$. For $j \in [\ell]$, $R_j \leftarrow \sum_{k \in [n]} R_{k,j}$, $\tilde{R}_j \leftarrow \sum_{k \in [i-1]} R_{k,j}$. If $R_j = 0_{\mathcal{R}} \vee \tilde{R}_j = 0_{\mathcal{R}}$, then return $(\perp, 0)$. $\widetilde{\text{apk}} \leftarrow \sum_{k \in [i-1]} \text{pk}_k$, $(R, \tilde{z}) \leftarrow \sigma_{i-1}$, $v \leftarrow H_{\text{non}}(m, \text{offs}, L)$. If $i = 1$, then $R \leftarrow \sum_{j \in [\ell]} v^{j-1} R_j$, $c \leftarrow H_{\text{ch}}(m, R, L)$. If $i > 1$, then If $R \neq \sum_{j \in [\ell]} v^{j-1} R_j$, then return $(\perp, 0)$. $\tilde{R} \leftarrow \sum_{j \in [\ell]} v^{j-1} \tilde{R}_j$, $c \leftarrow H_{\text{ch}}(m, R, L)$. If $\text{F}(\text{par}, \tilde{z}) \neq \tilde{R} + c \cdot \widetilde{\text{apk}}$, then return $(\perp, 0)$. $z_i \leftarrow c \cdot \text{sk}_i + \sum_{j \in [\ell]} v^{j-1} \cdot r_{i,j}$, $z \leftarrow \tilde{z} + z_i$, $\sigma_i \leftarrow (R, z)$. Return $(\sigma_i, 1)$.
Verify(pp, L, m, σ) : $(\text{pk}_1, \dots, \text{pk}_n) \leftarrow L$. For $i, j \in [n]$, If $\text{pk}_i = \text{pk}_j \wedge i \neq j$, then return 0. $(R, z) \leftarrow \sigma$, $c \leftarrow H_{\text{ch}}(m, R, L)$. If $\text{F}(\text{par}, z) = R + c \cdot \sum_{j \in [n]} \text{pk}_j$, then return 1. Otherwise, return 0.

Fig. 9. Our ordered multi-signature scheme $\text{OMS}_{\text{BDPT-H}}[\text{LHF}, \ell]$.

- **Initial Setup:** B_1 makes $2Q_s + 1$ queries to $\mathcal{O}^{\text{CH}}$. We denote the $2Q_s + 1$ outputs received from $\mathcal{O}^{\text{CH}}$ as $(\text{pk}^* = U_0, U_1, \dots, U_{2Q_s})$. Then, B_1 samples $t \xleftarrow{\$} [Q_k]$ and initializes $\text{ctr}_U \leftarrow 0$, $\text{ctr}_{\text{non}} \leftarrow 0$, $\text{ctr}_{\text{ch}} \leftarrow 0$, $\mathcal{K} \leftarrow \{\}$, $\mathcal{T} \leftarrow \{\}$, $\mathcal{Q} \leftarrow \{\}$, $\mathbb{T}^{H_{\text{non}}} \leftarrow \{\}$, $\mathbb{T}^{H_{\text{ch}}} \leftarrow \{\}$, and a flag $\text{win} \leftarrow 0$. Next, B_1 sets $\sigma_0 = (0_{\mathcal{R}}, 1_S)$ and $\text{pp} = (\text{par}, n, \sigma_0)$ and invokes A with an input pp .
- **Simulation of $\mathcal{O}^{H_{\text{non}}}$:** For a query (m, offs, L) by A where $\text{offs} = (R_{k,1}, R_{k,2})_{k \in [n]}$, B_1 checks a hash table $\mathbb{T}^{H_{\text{non}}}$. If $\mathbb{T}^{H_{\text{non}}}[m, \text{offs}, L] \neq \perp$, then B_1 returns $\mathbb{T}^{H_{\text{non}}}[m, \text{offs}, L]$ to A . Otherwise, B_1 updates $\text{ctr}_{\text{non}} \leftarrow \text{ctr}_{\text{non}} + 1$, computes $R = (\sum_{k \in [n]} R_{k,1}) + (v_{\text{ctr}_{\text{non}}} \cdot \sum_{k \in [n]} R_{k,2})$ where $v_{\text{ctr}_{\text{non}}}$ is the initial input element of B_1 . If $\sum_{k \in [n]} R_{k,2} = 0_{\mathcal{R}}$ or $\mathbb{T}^{H_{\text{ch}}}[m, R, L] \neq \perp$ holds, then B_1 aborts. Otherwise, B_1 makes a query to $\mathcal{O}^{H_{\text{ch}}}$ with (m, R, L) , updates $\mathbb{T}^{H_{\text{ch}}}[m, \text{offs}, L] \leftarrow v_{\text{ctr}_{\text{non}}}$, and returns $v_{\text{ctr}_{\text{non}}}$ to A .
- **Simulation of $\mathcal{O}^{H_{\text{ch}}}$:** For a query (m, R, L) by A , B_1 checks a hash table $\mathbb{T}^{H_{\text{ch}}}$. If $\mathbb{T}^{H_{\text{ch}}}[m, R, L] \neq \perp$, then B_1 returns $\mathbb{T}^{H_{\text{ch}}}[m, R, L]$ to A . Otherwise, B_1 updates $\text{ctr}_{\text{ch}} \leftarrow \text{ctr}_{\text{ch}} + 1$ and $\mathbb{T}^{H_{\text{ch}}}[m, R, L] \leftarrow c_{\text{ctr}_{\text{ch}}}$ and returns $c_{\text{ctr}_{\text{ch}}}$ to A , where $c_{\text{ctr}_{\text{ch}}}$ is the initial input element of B_1 .
- **Simulation of $\mathcal{O}^{\text{KeyReg}}$:** For a query (uid, aux) by A , if $\text{aux} = \epsilon$ holds or uid has been queried, B_1 proceeds as the actual $\mathcal{O}^{\text{KeyReg}}$ does. Otherwise, if this query is the t -th query with $\text{aux} = \epsilon$, then B_1 sets $\text{uid}^* \leftarrow \text{uid}$, $\mathcal{K}[\text{uid}^*] \leftarrow (1, \text{pk}^*, \perp)$, and returns pk^* to A . Otherwise, B_1 proceeds as the actual $\mathcal{O}^{\text{KeyReg}}$ does.
- **Simulation of $\mathcal{O}^{\text{SignOff}}$:** For a query $(\text{ssid}, \text{uid})$ by A , B_1 sets $(b, \text{pk}, \text{sk}) \leftarrow \mathcal{K}[\text{uid}]$. If $(b = 0 \vee \text{sk} = \epsilon) \vee (\mathcal{T}[\text{ssid}, \text{uid}] \neq \perp)$ hold, then B_1 returns $\perp$ to A . If $\text{uid} = \text{uid}^*$ holds, then B_1 sets $R_{i,1} \leftarrow U_{2\text{ctr}_U+1}$, $R_{i,2} \leftarrow U_{2\text{ctr}_U+2}$, $\text{off}_i \leftarrow (R_{i,1}, R_{i,2})$, $\text{ctr}_U \leftarrow \text{ctr}_U + 1$, and $\mathcal{T}[\text{ssid}, \text{uid}] \leftarrow (\text{off}_i, \perp)$ and returns off_i to A . If $\text{uid} \neq \text{uid}^*$ holds, then B_1 sets $r_{i,1} \xleftarrow{\$} \mathcal{D}$, $r_{i,2} \xleftarrow{\$} \mathcal{D}$, $R_{i,1} \leftarrow F(\text{par}, r_{i,1})$, $R_{i,2} \leftarrow F(\text{par}, r_{i,2})$, $\text{off}_i \leftarrow (R_{i,1}, R_{i,2})$, $\text{st}_i \leftarrow (r_{i,1}, r_{i,2})$, and $\mathcal{T}[\text{ssid}, \text{uid}] \leftarrow (\text{off}_i, \text{st}_i)$ and returns off_i to A .
- **Simulation of $\mathcal{O}^{\text{SignOn}}$:** For a query $(\text{ssid}, \text{uid}, L, m, \text{offs}, \sigma_{i-1}, i)$ by A , if this query is invalid (i.e., the query input does not satisfy the conditions of the first four “If” statements in $\mathcal{O}^{\text{SignOn}}$ of the EUF-CMA-KRM security game in Fig. 8), B_1 returns $(\perp, 0)$ to A . Let $(\text{pk}_1, \dots, \text{pk}_n) \leftarrow L$, $(R_{1,1}, R_{1,2}, \dots, R_{n,1}, R_{n,2}) \leftarrow \text{offs}$, $(R, \tilde{z}) \leftarrow \sigma_{i-1}$, $v \leftarrow H_{\text{non}}(m, \text{offs}, L)$, and $c \leftarrow H_{\text{ch}}(m, R, L)$. If $\text{uid} = \text{uid}^* \wedge \text{pk}^* = \text{pk}_i$ holds, then B_1 makes a query to $\mathcal{O}^{\text{Alnv}}$ with $Z_i = c \cdot \text{pk}_i + R_{i,1} + v \cdot R_{i,2}$ and obtains z_i such that $F(\text{par}, z_i) = Z_i$. If $\text{uid} \neq \text{uid}^* \vee \text{pk}^* \neq \text{pk}_i$ hold, then B_1 sets $(b, \text{pk}_i, \text{sk}_i) \leftarrow \mathcal{K}[\text{uid}]$, $(\text{off}_i, \text{st}_i) \leftarrow (\text{off}_i, \text{st}_i) \leftarrow \mathcal{T}[\text{ssid}, \text{uid}]$, and $z_i \leftarrow c \cdot \text{sk}_i + r_{i,1} + v \cdot r_{i,2}$. Finally, B sets $z \leftarrow \tilde{z} + z_i$ and $\sigma_i \leftarrow (R, z)$ and returns $(\sigma_i, 1)$ to A .
- **Final Procedure:** B_1 receives the forgery $(L^*, m^*, \sigma^* = (R^*, z^*))$ from A . If any of the validity checks in the EUF-CMA-KRM security game does not hold, then B_1 returns $(0, \perp, \perp)$ to A . If $\text{pk}^* \notin L^*$ or (m^*, L^*) is signed by $\mathcal{O}^{\text{SignOn}}$, then B_1 aborts. Otherwise, B_1 sets $\text{ctr}_{\text{non}}^*$ and ctr_{ch}^* as the values of ctr_{non} and ctr_{ch} at the moment $\mathbb{T}^{H_{\text{ch}}}[m^*, R^*, L^*]$ is defined. Next, B_1 sets an index j^* such that $\text{pk}^* = \text{pk}_{j^*} \in L^*$ and $K^* = \{\text{sk}_j\}_{j \in [n] \setminus \{j^*\}}$ where sk_j is a

secret key corresponding to the public key $\text{pk}_j \in L$ for $j \in [n] \setminus \{j^*\}$. Finally, B_1 outputs $(\text{ctr}_{\text{ch}}^*, \text{ctr}_{\text{non}}^*, \text{out} = (\text{ctr}_{\text{ch}}^*, \text{ctr}_{\text{non}}^*, j^*, L^*, K^*, R^*, c^* = c_{\text{ctr}_{\text{ch}}^*}, z^*))$ to its game and terminates.

The Algorithm $R_1^{\mathcal{O}^{\text{CH}}, \mathcal{O}^{\text{AInv}}(\cdot, \cdot, \cdot)}(\text{par})$. The algorithm R_1 takes a public parameter par of a linear hash function LHF as input. R_1 runs B_1 twice through the forking algorithm Fork^{B_1} and solves the AOMPR problem.

- **Initial Setup:** R_1 samples $v_1, \hat{v}_1, \dots, v_Q, \hat{v}_Q \xleftarrow{\$} \mathcal{S}(\text{par})^{2Q}$ and runs the forking algorithm $\text{Fork}^{B_1}(\text{pp}, v_1, \hat{v}_1, \dots, v_Q, \hat{v}_Q)$.
- **Oracle Queries During the Running of Fork^{B_1} :** In the first execution of B_1 , all queries to oracle $\mathcal{O}^{\text{CH}}$ from B_1 are relayed by R_1 to its own $\mathcal{O}^{\text{CH}}$ and returns an outputted value. Also, all queries to the oracle $\mathcal{O}^{\text{AInv}}$ from B_1 are relayed by R_1 to its own $\mathcal{O}^{\text{AInv}}$ and returns an outputted value. In the second execution of B_1 , for all queries to the oracle $\mathcal{O}^{\text{CH}}$ from B_1 , R_1 responds to them with the same values as the first execution. For queries to the oracle $\mathcal{O}^{\text{AInv}}$ from B_1 , if the concurrent counter value ctr_{ch} satisfies $\text{ctr}_{\text{ch}} < \text{ctr}_{\text{ch}}^*$, R_1 responds to them with the same values in the first execution (i.e., R_1 does not make a query to $\mathcal{O}^{\text{AInv}}$). If $\text{ctr}_{\text{ch}} \geq \text{ctr}_{\text{ch}}^*$ holds, then the queries to the oracle $\mathcal{O}^{\text{AInv}}$ from B_1 are relayed by R_1 to its own $\mathcal{O}^{\text{AInv}}$ and returns an outputted value.
- **Final Procedure:** R_1 receives a tuple $(b, \text{out}, \hat{\text{out}})$ from Fork^{B_1} . If $b = 0$ holds, then R_1 aborts. Otherwise, R_1 parses $(\text{ctr}_{\text{ch}}^*, \text{ctr}_{\text{non}}^*, j^*, L^*, K^*, R^*, c^*, z^*) \leftarrow \text{out}$ and $(\text{ctr}_{\text{ch}}^*, \text{ctr}_{\text{non}}^*, j^*, L^*, K^*, R^*, \hat{c}^*, \hat{z}^*) \leftarrow \hat{\text{out}}$. Note that $(\text{ctr}_{\text{ch}}^*, \text{ctr}_{\text{non}}^*, j^*, L^*, K^*, R^*)$ from two outputs are identical due to the way R_1 simulates the view and the resulting output by Fork^{B_1} with $b = 1$. Next, R_1 computes $u_0 \leftarrow (z^* - \hat{z}^*) \cdot (c^* - \hat{c}^*)^{-1} - \sum_{j \in [n], i \neq j^*} \text{sk}_j$ where $K^* = \{\text{sk}_j\}_{j \in [n] \setminus \{j^*\}}$. For each (U_{2j+1}, U_{2j+2}) where $j \in [Q_s]$, R_1 extracts (u_{2j+1}, u_{2j+2}) by dividing it into the following three cases:
 1. The case that Fork^{B_1} queries twice to $\mathcal{O}^{\text{AInv}}$ with $Z_i = c \cdot \text{pk}_i + U_{2j+1} + v \cdot U_{2j+2}$ and $Z'_i = c' \cdot \text{pk}_i + U_{2j+1} + v' \cdot U_{2j+2}$.
 2. The case that Fork^{B_1} makes a query only once to $\mathcal{O}^{\text{AInv}}$ with $Z_i = c \cdot \text{pk}_i + U_{2j+1} + v \cdot U_{2j+2}$.
 3. The case that Fork^{B_1} does not query to $\mathcal{O}^{\text{AInv}}$ with $Z_i = c \cdot \text{pk}_i + U_{2j+1} + v \cdot U_{2j+2}$.

In Case 1., R_1 knows u_0 and (z_i, z'_i) such that $\text{pk}_i = \text{pk}^* = U_0 = F(\text{par}, u_0)$, $Z_i = F(\text{par}, z_i)$, and $Z'_i = F(\text{par}, z'_i)$. If $v \neq v'$ holds, then R_1 extracts (u_{2j+1}, u_{2j+2}) by solving the following equations

$$\begin{cases} z_i = c \cdot u_0 + u_{2j+1} + v \cdot u_{2j+2} \\ z'_i = c' \cdot u_0 + u_{2j+1} + v' \cdot u_{2j+2} \end{cases}.$$

Otherwise, R_1 aborts.

In Case 2., R_1 knows u_0 and z_i such that $\text{pk}_i = \text{pk}^* = U_0 = F(\text{par}, u_0)$ and $Z_i = F(\text{par}, z_i)$. R_1 queries U_{2j+1} to $\mathcal{O}^{\text{AInv}}$ and obtains u_{2j+1} . If $v \neq 0$, R_1

extracts u_{2j+2} by solving the following equation $z_i = c \cdot u_0 + u_{2j+1} + v \cdot u_{2j+2}$. Otherwise, R_1 aborts.

In Case 3., R_1 queries U_{2j+1} and U_{2j+2} to $\mathcal{O}^{\text{AInv}}$ and obtains (u_{2j+1}, u_{2j+2}) .

If R_1 fails to extract $(u_0, \dots, u_{2Q_s+2})$, then R_1 aborts. Finally, R_1 outputs the solution $(u_0, \dots, u_{2Q_s+2})$ for the AOMPR problem.

Probability Analysis for R_1 . First, we give the probability analysis for B_1 . To this end, we consider the probability that B_1 aborts in the simulation of the game. In the simulation of $\mathcal{O}^{H_{\text{non}}}$, if $\sum_{k \in [n]} R_{k,2} = 0_{\mathcal{R}}$ or $\mathbb{T}^{H^{\text{ch}}}[m, R, L] \neq \perp$ holds, then B_1 aborts where $R = (\sum_{k \in [n]} R_{k,1}) + (v_{\text{ctr}_{\text{non}}} \cdot \sum_{k \in [n]} R_{k,2})$. For each query, this occurs with probability at most $Q/|\mathcal{R}(\text{par})|$. By taking union bound, the probability that B_1 aborts in the simulation is at most $Q^2/|\mathcal{R}(\text{par})|$. Also, we consider the probability that B_1 (does not abort the simulation of the game and) aborts in the final procedure. Recall that B_1 aborts in the final procedure if $\text{pk}^* \notin L^*$ or (m^*, L^*) is signed by $\mathcal{O}^{\text{SignOn}}$. This probability is bounded by $1/Q_k$. Thus, we see that the probability that B_1 outputs $(\text{ctr}_{\text{ch}}^*, \text{ctr}_{\text{non}}^*, \text{out})$ with $\text{ctr}_{\text{ch}}^* \neq 0$ is bounded by $\left(\text{Adv}_{\text{OMS}_{\text{BDPT}}[\text{LHF}, 2], \text{A}}^{\text{EUF-CMA-KRM}}(\lambda) - Q^2/|\mathcal{R}(\text{par})|\right)/Q_k$.

Next, we consider the probability that R_1 extracts the solution for the AOMPR problem. If R_1 receives $(0, \text{out}, \hat{\text{out}})$, then R_1 aborts. From the forking lemma, the probability that R_1 does not abort in this step is bounded by $\text{frk} \geq \text{acc} \cdot \left(\frac{\text{acc}}{Q} - \frac{1}{|\mathcal{S}(\text{par})|}\right)$ where $\text{acc} = \left(\text{Adv}_{\text{OMS}_{\text{BDPT}}[\text{LHF}, 2], \text{A}}^{\text{EUF-CMA-KRM}}(\lambda) - Q^2/|\mathcal{R}(\text{par})|\right)/Q_k$. For each (U_{2j+1}, U_{2j+2}) where $j \in [Q_s]$, the probability that R_1 fails to extract (u_{2j+1}, u_{2j+2}) is bounded by $1/|\mathcal{S}(\text{par})|$. By taking the union bound, we have

$$\text{Adv}_{\text{LHF}, R_1}^{\text{AOMPR}}(\lambda) \geq \left(\frac{\text{acc}}{Q} - \frac{1}{|\mathcal{S}(\text{par})|}\right) - \frac{Q_s}{|\mathcal{S}(\text{par})|}.$$

Reduction against the Type 2 Adversary. Let A be a type 2 adversary for the EUF-CMA-KRM security game $\text{G}_{\text{OMS}_{\text{BDPT}}[\text{LHF}, 2], \text{A}}^{\text{EUF-CMA-KRM}}$ that receives at most Q_k public keys from $\mathcal{O}^{\text{KeyReg}}$, makes at most Q_h queries to the random oracle, starts at most Q_s sessions in total with $\mathcal{O}^{\text{SignOff}}$, makes a query $(L^*, m^*, \text{offs}^*, \sigma_{i^*-1}^*, i^*)$ to $\mathcal{O}^{\text{SignOn}}$, and the flag win is changed to 1. Let $Q = Q_s + 2Q_h + 1$.

We construct the reduction R_2 by following two steps. In the first step, we construct an algorithm $B_2^{\mathcal{O}^{\text{CH}}, \mathcal{O}^{\text{AInv}}, \text{A}(\cdot)}$ which is obtained with some modification to B_1 . In the second step, we use B_2 to construct the reduction R_2 which runs the forking algorithm Fork^{B_2} and extracts the preimage of LHF. Now, we give the detail of B_2 and R_2 .

The Algorithm $B_2^{\mathcal{O}^{\text{CH}}, \mathcal{O}^{\text{AInv}}, \text{A}(\cdot)}$ $(\text{par}, v_1, \dots, v_Q, c_1, \dots, c_Q)$. The algorithm B_2 takes a tuple $(\text{par}, v_1, \dots, v_Q, c_1, \dots, c_Q)$ as input, where par is a public parameter of LHF and $(v_1, \dots, v_Q, c_1, \dots, c_Q) \in \mathcal{S}(\text{par})^{2Q}$. B_2 simulates the game $\text{G}_{\text{LHF}, \text{A}}^{\text{AOMPR}}$ to A by accessing to the oracles $\mathcal{O}^{\text{CH}}$ and $\mathcal{O}^{\text{AInv}}$ that are defined in the same way as the game $\text{G}_{\text{LHF}, \text{A}}^{\text{AOMPR}}$.

- **Initial Setup:** B_2 follows the same initial setup procedure as B_1 , except for the following: It samples $s \xleftarrow{\$} [Q_k] \setminus \{t\}$. An index s indicates the honest public key of the owner who is responsible for detecting a type 2 forgery.
- **Simulation of $\mathcal{O}^{H_{\text{non}}}$, $\mathcal{O}^{H_{\text{ch}}}$, $\mathcal{O}^{\text{SignOff}}$:** B_2 follows the same procedure as B_1 .
- **Simulation of $\mathcal{O}^{\text{KeyReg}}$:** B_2 follows the same initial setup procedure as B_1 except for the following: If A makes the s -th query on (uid, aux) with $\text{aux} = \epsilon$, then B_1 memorizes an honest key pk' .
- **Simulation of $\mathcal{O}^{\text{SignOn}}$:** For a query $(\text{ssid}, \text{uid}, L^*, m^*, \text{offs}^*, \sigma_{i^*-1}^*, i^*)$, B_2 does the same procedure as B_1 except for the following:
 - Let $L^* = (\text{pk}_1, \dots, \text{pk}_n)$, $\text{offs}^* = (R_{1,1}^*, R_{1,2}^*, \dots, R_{n,1}^*, R_{n,2}^*)$, and $\sigma_{i^*-1}^* = (R^*, z^*)$. B_2 computes $\tilde{R}_1 \leftarrow \sum_{k \in [i^*-1]} R_{k,1}$, $\tilde{R}_2 \leftarrow \sum_{k \in [i^*-1]} R_{k,2}$, $v^* \leftarrow H_{\text{ch}}(m^*, \text{offs}^*, L^*)$, $\tilde{R}^* \leftarrow \tilde{R}_1 + v^* \tilde{R}_2$, and $c^* \leftarrow H_{\text{ch}}(m^*, R^*, L^*)$. If $F(\text{par}, z^*) = \tilde{R} + c^* \sum_{k \in [i-1]} \text{pk}_k$ holds, then B_2 proceed as follows: If $\text{pk}_{i^*} \neq \text{pk}'$ or there is no $j^* < i^*$ such that $\text{pk}_{j^*} = \text{pk}^*$ or (m^*, L^*) have not been signed with pk^* , then B_2 aborts. Otherwise, B_2 sets j^* such that $\text{pk}_{j^*} = \text{pk}^*$. B_2 sets $\text{ctr}_{\text{non}}^*$ and ctr_{ch}^* as the values of ctr_{non} and ctr_{ch} at the moment $\mathbb{T}^{H_{\text{ch}}}[m^*, R^*, L^*]$ is defined. B_1 sets an index j^* such that $\text{pk}^* = \text{pk}_{j^*} \in L^*$ and $K^* = \{\text{sk}_j\}_{j \in [n] \setminus \{j^*\}}$ where sk_j is a secret key corresponding to the public key $\text{pk}_j \in L$ for $j \in [n] \setminus \{j^*\}$. Finally, B_1 outputs $(\text{ctr}_{\text{ch}}^*, \text{ctr}_{\text{non}}^*, \text{out} = (\text{ctr}_{\text{ch}}^*, \text{ctr}_{\text{non}}^*, i^*, j^*, L^*, K^*, \tilde{R}^*, c^*, z^*))$ and terminates.

The Algorithm $R_2^{\mathcal{O}^{\text{CH}}(), \mathcal{O}^{\text{Alnv}}(\cdot, \cdot, \cdot)}(\text{par})$. The algorithm R_2 takes a public parameter par of a linear hash function LHF as input. R_2 runs B_2 twice through the forking algorithm Fork^{B_2} and solves the AOMPR problem as follows.

- **Initial Setup:** R_2 samples $v_1, \hat{v}_1, \dots, v_Q, \hat{v}_Q \xleftarrow{\$} \mathcal{S}(\text{par})^{2Q}$ and runs the forking algorithm $\text{Fork}^{B_2}(\text{pp}, v_1, \hat{v}_1, \dots, v_Q, \hat{v}_Q)$.
- **Oracle Queries During the Running of Fork^{B_2} :** R_2 follows the same initial setup procedure as R_1 .
- **Final Procedure:** R_2 receives a tuple $(b, \text{out}, \hat{\text{out}})$ from Fork^{B_2} . If $b = 0$ holds, then R_2 aborts. Otherwise, R_2 parses $(\text{ctr}_{\text{ch}}^*, \text{ctr}_{\text{non}}^*, i^*, j^*, L^*, K^*, \tilde{R}^*, c^*, z^*) \leftarrow \text{out}$ and $(\text{ctr}_{\text{ch}}^*, \text{ctr}_{\text{non}}^*, i^*, j^*, L^*, K^*, \tilde{R}^*, \hat{c}^*, \hat{z}^*) \leftarrow \hat{\text{out}}$. Note that $(\text{ctr}_{\text{ch}}^*, \text{ctr}_{\text{non}}^*, j^*, L^*, K^*, R^*)$ from two outputs are identical due to the way R_2 simulates the view and the resulting output by Fork^{B_2} with $b = 1$. Then, R_2 computes $u_0 \leftarrow (z^* - \hat{z}^*) \cdot (c^* - \hat{c}^*)^{-1} - \sum_{j \in [n], i \neq j^*} \text{sk}_j$ where $K^* = \{\text{sk}_j\}_{j \in [n] \setminus \{j^*\}}$. For each (U_{2j+1}, U_{2j+2}) where $j \in [Q_s]$, R_2 extracts (u_{2j+1}, u_{2j+2}) by following the same procedure as R_1 . If R_2 fails to extract $(u_0, \dots, u_{2Q_s+2})$, then R_2 aborts. Finally, R_2 outputs the solution $(u_0, \dots, u_{2Q_s+2})$ for the AOMPR problem.

Probability Analysis of R_2 . The probability analysis for B_2 is obtained in the same way as B_1 except that the index s is introduced to detect the Type 2 forgery. By considering the probability that B_2 fails to guess s , we see that

the probability that B_2 outputs $(\text{ctr}_{\text{ch}}^*, \text{ctr}_{\text{non}}^*, \text{out})$ with $\text{ctr}_{\text{ch}}^* \neq 0$ is bounded by $\left(\text{Adv}_{\text{OMS}_{\text{BDPT}}[\text{LHF}, 2], \text{A}}^{\text{EUF-CMA-KRM}}(\lambda) - Q^2/|\mathcal{R}(\text{par})|\right)/Q_k^2$.

Also, the probability analysis for R_2 is obtained in the same way as R_1 . Thus, we have

$$\text{Adv}_{\text{LHF}, R_2}^{\text{AOMPR}}(\lambda) \geq \left(\frac{\text{acc}}{Q} - \frac{1}{|S(\text{par})|}\right) - \frac{Q_s}{|S(\text{par})|},$$

where $\text{acc} = \left(\text{Adv}_{\text{OMS}_{\text{BDPT}}[\text{LHF}, 2], \text{A}}^{\text{EUF-CMA-KRM}}(\lambda) - Q^2/|\mathcal{R}(\text{par})|\right)/Q_k^2$.

From above facts, we conclude Theorem 1.

□ (**Theorem 1**)

Combining Lemma 3 with Theorem 1, we have the following fact.

Corollary 1. *If the DL assumption holds, then the scheme $\text{OMS}_{\text{BDPT-H}}[\text{LHF}_{\text{Ped}}, \ell = 2]$ satisfies the EUF-CMA-KRM security in the ROM.*

References

1. Handan Kiling Alper and Jeffrey Burdges. Two-round trip schnorr multi-signatures via delinearized witnesses. In *CRYPTO 2021, Part I*, volume 12825 of *LNCS*, pages 157–188. Springer, 2021.
2. Renas Bacho and Benedikt Wagner. Tightly secure non-interactive BLS multi-signatures. In *ASIACRYPT 2024, Part II*, volume 15485 of *LNCS*, pages 397–422. Springer, 2024.
3. Renas Bacho and Benedikt Wagner. T-spoon: Tightly secure two-round multi-signatures with key aggregation. In *CRYPTO 2025, Part VI*, volume 16005 of *LNCS*, pages 256–290. Springer, 2025.
4. Ali Bagherzandi, Jung Hee Cheon, and Stanislaw Jarecki. Multisignatures secure under the discrete logarithm assumption and a generalized forking lemma. In *ACM CCS 2008*, pages 449–458. ACM, 2008.
5. Carsten Baum, Bernardo David, Elena Pagnin, and Akira Takahashi. Universally composable interactive and ordered multi-signatures. In *PKC 2025, Part II*, volume 15675 of *LNCS*, pages 3–31. Springer, 2025.
6. Carsten Baum, Bernardo Machado David, Elena Pagnin, and Akira Takahashi. Cascade: (time-based) cryptography from space communications delay. In *SCN 2024, Part I*, volume 14973 of *LNCS*, pages 252–274. Springer, 2024.
7. Mihir Bellare and Wei Dai. Chain reductions for multi-signatures and the HBMS scheme. In *ASIACRYPT 2021, Part IV*, volume 13093 of *LNCS*, pages 650–678. Springer, 2021.
8. Mihir Bellare, Chanathip Namprempre, and Gregory Neven. Unrestricted aggregate signatures. In *ICALP 2007*, volume 4596 of *LNCS*, pages 411–422. Springer, 2007.
9. Mihir Bellare and Gregory Neven. Multi-signatures in the plain public-key model and a general forking lemma. In *ACM CCS 2006*, pages 390–399. ACM, 2006.
10. Alexandra Boldyreva, Craig Gentry, Adam O’Neill, and Dae Hyun Yum. Ordered multisignatures and identity-based sequential aggregate signatures, with applications to secure routing. In *ACM CCS 2007*, pages 276–285. ACM, 2007.
11. Dan Boneh, Manu Drijvers, and Gregory Neven. Compact multi-signatures for smaller blockchains. In *ASIACRYPT 2018, Part II*, volume 11273 of *LNCS*, pages 435–464. Springer, 2018.

12. Dan Boneh, Craig Gentry, Ben Lynn, and Hovav Shacham. Aggregate and verifiably encrypted signatures from bilinear maps. In *EUROCRYPT 2003*, volume 2656 of *LNCS*, pages 416–432. Springer, 2003.
13. Dan Boneh, Aditi Partap, and Brent Waters. Accountable multi-signatures with constant size public keys. In *PKC 2025, Part II*, volume 15675 of *LNCS*, pages 32–65. Springer, 2025.
14. Yanbo Chen and Yunlei Zhao. Half-aggregation of schnorr signatures with tight reductions. In *ESORICS 2022, Part II*, volume 13555 of *LNCS*, pages 385–404. Springer, 2022.
15. Elizabeth C. Crites, Chelsea Komlo, and Mary Maller. How to prove schnorr assuming schnorr: Security of multi- and threshold signatures. *IACR Cryptol. ePrint Arch.*, page 1375, 2021.
16. Manu Drijvers, Kasma Edalatnejad, Bryan Ford, Eike Kiltz, Julian Loss, Gregory Neven, and Igors Stepanovs. On the security of two-round multi-signatures. In *IEEE SP 2019*, pages 1084–1101. IEEE, 2019.
17. Marc Fischlin, Anja Lehmann, and Dominique Schröder. History-free sequential aggregate signatures. In *SCN 2012*, volume 7485 of *LNCS*, pages 113–130. Springer, 2012.
18. Masayuki Fukumitsu and Shingo Hasegawa. A tightly secure ddh-based multisignature with public-key aggregation. *Int. J. Netw. Comput.*, 11(2):319–337, 2021.
19. Kazuharu Itakura and Katsuhiko Nakamura. A public-key cryptosystem suitable for digital multisignatures. *NEC Research & Development*, 71:1–8, 1983.
20. Kwangsu Lee, Dong Hoon Lee, and Moti Yung. Aggregating cl-signatures revisited: Extended functionality and better efficiency. In *FC 2013*, volume 7859 of *LNCS*, pages 171–188. Springer, 2013.
21. Steve Lu, Rafail Ostrovsky, Amit Sahai, Hovav Shacham, and Brent Waters. Sequential aggregate signatures and multisignatures without random oracles. In *EUROCRYPT 2006*, volume 4004 of *LNCS*, pages 465–485. Springer, 2006.
22. Steve Lu, Rafail Ostrovsky, Amit Sahai, Hovav Shacham, and Brent Waters. Sequential aggregate signatures, multisignatures, and verifiably encrypted signatures without random oracles. *J. Cryptol.*, 26(2):340–373, 2013.
23. Anna Lysyanskaya, Silvio Micali, Leonid Reyzin, and Hovav Shacham. Sequential aggregate signatures from trapdoor permutations. In *EUROCRYPT 2004*, volume 3027 of *LNCS*, pages 74–90. Springer, 2004.
24. Changshe Ma, Jian Weng, Yingjiu Li, and Robert H. Deng. Efficient discrete logarithm based multi-signature scheme in the plain public key model. *Des. Codes Cryptogr.*, 54(2):121–133, 2010.
25. Gregory Maxwell, Andrew Poelstra, Yannick Seurin, and Pieter Wuille. Simple schnorr multi-signatures with applications to bitcoin. *IACR Cryptol. ePrint Arch.*, page 68, 2018. Version 0180118:124757.
26. Gregory Maxwell, Andrew Poelstra, Yannick Seurin, and Pieter Wuille. Simple schnorr multi-signatures with applications to bitcoin. *Des. Codes Cryptogr.*, 87(9):2139–2164, 2019.
27. Jonas Nick, Tim Ruffing, and Yannick Seurin. Musig2: Simple two-round schnorr multi-signatures. In *CRYPTO 2021, Part I*, volume 12825 of *LNCS*, pages 189–221. Springer, 2021.
28. Jonas Nick, Tim Ruffing, and Yannick Seurin. Musig2: Simple two-round schnorr multi-signatures. *IACR Cryptol. ePrint Arch.*, page 1261, 2021. Version 20210706:150749.

29. Jiaxin Pan and Benedikt Wagner. Chopsticks: Fork-free two-round multi-signatures from non-interactive assumptions. In *EUROCRYPT 2023, Part V*, volume 14008 of *LNCS*, pages 597–627. Springer, 2023.
30. Jiaxin Pan and Benedikt Wagner. Toothpicks: More efficient fork-free two-round multi-signatures. In *EUROCRYPT 2024, Part I*, volume 14651 of *LNCS*, pages 460–489. Springer, 2024.
31. Torben P. Pedersen. Non-interactive and information-theoretic secure verifiable secret sharing. In *CRYPTO '91*, volume 576 of *LNCS*, pages 129–140. Springer, 1991.
32. David Pointcheval and Olivier Sanders. Short randomizable signatures. In *CT-RSA 2016*, volume 9610 of *LNCS*, pages 111–126. Springer, 2016.
33. Ewa Syta, Iulia Tamas, Dylan Visher, David Isaac Wolinsky, Philipp Jovanovic, Linus Gasser, Nicolas Gailly, Ismail Khoffi, and Bryan Ford. Keeping authorities “honest or bust” with decentralized witness cosigning. In *IEEE SP 2016*, pages 526–545. IEEE Computer Society, 2016.
34. Kaoru Takemure, Yusuke Sakai, Bagus Santoso, Goichiro Hanaoka, and Kazuo Ohta. More efficient two-round multi-signature scheme with provably secure parameters for standardized elliptic curves. *IEICE Trans. Fundam. Electron. Commun. Comput. Sci.*, 107(7):966–988, 2024.
35. Stefano Tessaro and Chenzhi Zhu. Threshold and multi-signature schemes from linear hash functions. In *EUROCRYPT 2023, Part V*, volume 14008 of *LNCS*, pages 628–658. Springer, 2023.
36. Masayuki Tezuka and Keisuke Tanaka. Ordered multi-signatures with public-key aggregation from sxdh assumption, 2025.
37. Jianghong Wei, Guohua Tian, Ding Wang, Fuchun Guo, Willy Susilo, and Xiaofeng Chen. Pixel+ and pixel++: Compact and efficient forward-secure multi-signatures for pos blockchain consensus. In *USENIX Security 2024*. USENIX Association, 2024.
38. Naoto Yanai, Eikoh Chida, Masahiro Mambo, and Eiji Okamoto. A cdh-based ordered multisignature scheme provably secure without random oracles. *J. Inf. Process.*, 22(2):366–375, 2014.
39. Naoto Yanai, Masahiro Mambo, and Eiji Okamoto. An ordered multisignature scheme under the CDH assumption without random oracles. In *ISC 2013*, volume 7807 of *LNCS*, pages 367–377. Springer, 2013.
40. Naoto Yanai, Masahiro Mambo, and Eiji Okamoto. A cdh-based ordered multisignature scheme in the standard model with better efficiency. In *ISITA 2014*, pages 236–240. IEEE, 2014.

A The OMDL Assumption and Its Variant

Definition 8 (OMDL assumption). Let GrGen be a cyclic group generator and A be a PPT adversary. The one-more discrete logarithm (OMDL) assumption on GrGen is defined by the OMDL game $\mathcal{G}_{\text{GrGen},A}^{\text{OMDL}}$ in Fig. 10. The advantage of A is defined as $\text{Adv}_{\text{GrGen},A}^{\text{OMDL}}(\lambda) := \Pr[\mathcal{G}_{\text{GrGen},A}^{\text{OMDL}}(1^\lambda) \Rightarrow 1]$. The OMDL assumption holds on GrGen if for any PPT adversary A , $\text{Adv}_{\text{GrGen},A}^{\text{OMDL}}(\lambda)$ is $\text{negl}(\lambda)$.

Definition 9 (AOMDL assumption [27]). Let GrGen be a cyclic group generator and A be a PPT adversary. The algebraic one-more discrete logarithm (AOMDL) assumption on GrGen is defined by the AOMDL game $\mathcal{G}_{\text{GrGen},A}^{\text{AOMDL}}$ in Fig. 10. The advantage of A is defined as $\text{Adv}_{\text{GrGen},A}^{\text{AOMDL}}(\lambda) := \Pr[\mathcal{G}_{\text{GrGen},A}^{\text{AOMDL}}(1^\lambda) \Rightarrow 1]$. The AOMDL assumption holds on GrGen if for any PPT adversary A , $\text{Adv}_{\text{GrGen},A}^{\text{AOMDL}}(\lambda)$ is $\text{negl}(\lambda)$.

GAME $\mathcal{G}_{\text{GrGen},A}^{\text{OMDL}}(1^\lambda), \mathcal{G}_{\text{GrGen},A}^{\text{AOMDL}}(1^\lambda)$: $\text{par} = (\mathbb{G}, p, g) \leftarrow \text{GrGen}(1^\lambda), q_{\text{CH}} \leftarrow 0, q_{\text{DL}} \leftarrow 0, x_0 \xleftarrow{\\$} \mathbb{Z}_p, X_0 \leftarrow g^{x_0},$ $(x'_0, \dots, x'_{q_{\text{CH}}}) \leftarrow A^{\mathcal{O}^{\text{CH}}(\cdot), \mathcal{O}^{\text{DL}}(\cdot)}(\text{par}, X_0).$ // $\mathcal{G}_{\text{GrGen},A}^{\text{OMDL}}$ $(x'_0, \dots, x'_{q_{\text{CH}}}) \leftarrow A^{\mathcal{O}^{\text{CH}}(\cdot), \mathcal{O}^{\text{ADL}}(\cdot, \cdot)}(\text{par}, X_0).$ // $\mathcal{G}_{\text{GrGen},A}^{\text{AOMDL}}$ If $(x_0, \dots, x_{q_{\text{CH}}}) = (x'_0, \dots, x'_{q_{\text{CH}}}) \wedge q_{\text{CH}} \geq q_{\text{DL}}$, return 1. Otherwise, return 0.
Oracle $\mathcal{O}^{\text{CH}}()$: $q_{\text{CH}} \leftarrow q_{\text{CH}} + 1, x_{q_{\text{CH}}} \xleftarrow{\\$} \mathbb{Z}_p, X_{q_{\text{CH}}} \leftarrow g^{x_{q_{\text{CH}}}},$ return $X_{q_{\text{CH}}}$.
Oracle $\mathcal{O}^{\text{DL}}(X)$: $q_{\text{DL}} \leftarrow q_{\text{DL}} + 1,$ return $x \leftarrow \log_g X$.
Oracle $\mathcal{O}^{\text{ADL}}(X, (\alpha, (\beta_i)_{i \in \{0, \dots, q_{\text{CH}}\}}))$: If $X \neq g^\alpha \prod_{i \in \{0, \dots, q_{\text{CH}}\}} X_i^{\beta_i}$, return $\perp$. $q_{\text{DL}} \leftarrow q_{\text{DL}} + 1,$ return $x \leftarrow \alpha + \sum_{i \in \{0, \dots, q_{\text{CH}}\}} \beta_i x_i$.

Fig. 10. The OMDL game $\mathcal{G}_{\text{GrGen},A}^{\text{OMDL}}$ and the AOMDL game $\mathcal{G}_{\text{GrGen},A}^{\text{AOMDL}}$. The difference between these games is highlighted with boxes. $\mathcal{G}_{\text{GrGen},A}^{\text{OMDL}}$ is defined by white box and $\mathcal{G}_{\text{GrGen},A}^{\text{AOMDL}}$ is defined by coral color box.

We note that the OMDL assumption implies the AOMDL assumption and the AOMDL assumption implies the DL assumption. However, it is not clear that the AOMDL assumption implies the OMDL assumption. Also, it is not clear that the DL assumption implies the DL assumption,

B The Baum et al.'s Scheme OMS_{BDPT}

The ordered multi-signature scheme OMS_{BDPT} [5] in the key registration model is given in Fig. 11. The security of $\text{OMS}_{\text{BDPT}}[\ell]$ with $\ell = 2$ is proven under the DL assumption.

$\text{Setup}(1^\lambda) :$ $\text{par} = (\mathbb{G}, p, g) \leftarrow \text{GrGen}(1^\lambda), n = \text{poly}(\lambda),$ choose hash functions $H_{\text{non}}, H_{\text{ch}} : \{0, 1\}^* \rightarrow \mathbb{Z}_p, \sigma_0 = (1_{\mathbb{G}}, 0).$ Return $\text{pp} \leftarrow (\text{par}, n, H_{\text{non}}, H_{\text{ch}}, \sigma_0).$
$\text{KeyGen}(\text{pp}) :$ $\text{sk} \xleftarrow{\$} \mathbb{Z}_p, \text{pk} \leftarrow g^{\text{sk}}.$ Return $(\text{pk}, \text{sk}).$
$\text{SignOff}(\text{pp}, \text{sk}_i) :$ For $j \in [\ell], r_{i,j} \xleftarrow{\$} \mathbb{Z}_p, R_{i,j} \leftarrow g^{r_{i,j}}.$ Return $(\text{off}_i, \text{st}_i) \leftarrow ((R_{i,1}, \dots, R_{i,\ell}), (r_{i,1}, \dots, r_{i,\ell})).$
$\text{SignOn}(\text{pp}, \text{sk}, \text{st}_i, L, m, \text{offs} = (\text{off}_k)_{k \in [n]}, \sigma_{i-1}) :$ $(\text{pk}_1, \dots, \text{pk}_n) \leftarrow L, (r_{i,1}, \dots, r_{i,\ell}) \leftarrow \text{st}_i, ((R_{k,1}, \dots, R_{k,\ell})_{k \in [n]}) \leftarrow \text{offs}.$ For $j \in [\ell],$ $R_j \leftarrow \prod_{k \in [n]} R_{k,j}, \tilde{R}_j \leftarrow \prod_{k \in [i-1]} R_{k,j},$ If $R_j = 1_{\mathbb{G}} \vee \tilde{R}_j = 1_{\mathbb{G}},$ return $(\perp, 0).$ $\widetilde{\text{apk}} \leftarrow \prod_{k \in [i-1]} \text{pk}_k, (R, \tilde{z}) \leftarrow \sigma_{i-1}, v \leftarrow H_{\text{non}}(m, \text{offs}, L).$ If $i = 1, R \leftarrow \prod_{j \in [\ell]} R_j^{v^{j-1}}, c \leftarrow H_{\text{ch}}(m, R, L).$ If $i > 1,$ If $R \neq \prod_{j \in [\ell]} R_j^{v^{j-1}},$ return $(\perp, 0).$ $\tilde{R} \leftarrow \prod_{j \in [\ell]} \tilde{R}_j^{v^{j-1}}, c \leftarrow H_{\text{ch}}(m, R, L),$ If $g^{\tilde{z}} \neq \tilde{R} \cdot (\widetilde{\text{apk}})^c,$ return $(\perp, 0).$ $z_i \leftarrow c \cdot \text{sk}_i + \sum_{j \in [\ell]} v^{j-1} \cdot r_{i,j}, z \leftarrow \tilde{z} + z_i, \sigma_i \leftarrow (R, z)$ Return $(\sigma_i, 1).$
$\text{Verify}(\text{pp}, L, m, \sigma) :$ $(\text{pk}_1, \dots, \text{pk}_n) \leftarrow L.$ For $i, j \in [n],$ if $\text{pk}_i = \text{pk}_j \wedge i \neq j,$ return 0. $(R, z) \leftarrow \sigma, c \leftarrow H_{\text{ch}}(m, R, L).$ If $g^z = R \cdot \prod_{j \in [n]} \text{pk}_j^c,$ return 1. Otherwise, return 0.

Fig. 11. The ordered multi-signature scheme $\text{OMS}_{\text{BDPT}}[\ell]$.

Theorem 2 ([5]). $\text{OMS}_{\text{BDPT}}[2]$ satisfies the EUF-CMA-KRM security under the OMDL assumption in the ROM.